

PROJECT: AWS EC2 Auto Deployment using Docker

Services: EC2, Jenkins, Docker Project

Goal: automate the process of building, storing, and deploying an application using a CI/CD pipeline so that whenever code is updated, the application is automatically deployed on an AWS EC2 server using Docker containers.

Skills: CI/CD Pipeline Implementation, Containerization, Docker Image Registry management.

Definition:

AWS EC2 Auto Deployment using Docker is a DevOps approach that automates application deployment in the cloud. It uses Amazon EC2 for hosting, Docker for containerization, and Jenkins for CI/CD automation. When code is pushed to GitHub, Jenkins builds a Docker image and deploys it automatically on EC2. This process reduces manual work, ensures consistency, and enables faster and reliable deployments.

Scope of the Project:

The scope of this project is to design and implement an automated deployment system using cloud and DevOps technologies. It involves integrating Amazon EC2, Docker, and Jenkins to build, store, and deploy applications through a CI/CD pipeline. The project includes creating Docker images, pushing them to a registry like Docker Hub, and automatically deploying containers on EC2 instances. It aims to improve deployment speed, reduce human errors, and ensure consistency. The solution can be extended for scalable cloud applications and microservices architecture.

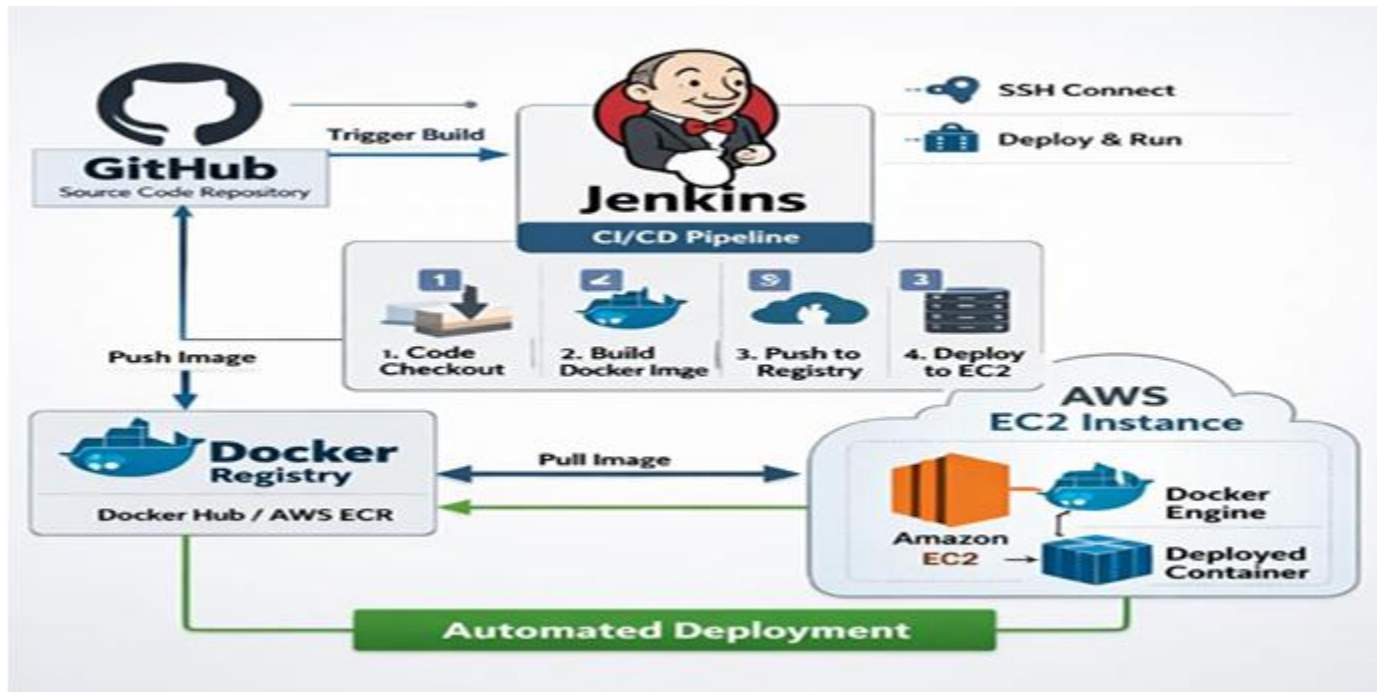
Methodologies:

1. The project follows the **CI/CD (Continuous Integration and Continuous Deployment)** approach for automation.
2. Code changes pushed to GitHub trigger automated build and deployment using Jenkins.
3. The application is containerized using Docker to ensure consistency across environments.
4. Docker images are stored in a registry like Docker Hub for easy access and version control.
5. The application is deployed automatically on Amazon EC2 without manual intervention.
6. This methodology reduces errors, speeds up deployment, and supports scalable cloud applications.

Services Used in this Project:

1. Amazon EC2 – Provides a virtual server in the cloud to host and run the application and Docker containers.
2. Jenkins – Acts as the CI/CD tool to automate building, testing, and deployment of the application.
3. Docker – Used for containerizing the application to ensure it runs consistently across environments.
4. Docker Hub – Stores and manages Docker images so they can be easily pulled and deployed.
5. GitHub – Hosts the source code repository and triggers the CI/CD pipeline when changes are pushed.

Architecture Diagram:



Implementation and Execution of AWS EC2 Auto Deployment using Docker.

STEP 1: Create AWS EC2 Instance

- Launch an Amazon EC2 instance in AWS.
- Choose Amazon Linux AMI.
- Configure security group (ports 22, 8080, 8081).

The screenshot shows the AWS Management Console for EC2 Instances. On the left, there's a navigation menu with 'EC2' selected. The main area shows a table of instances. One instance, 'jenkins-server', is selected and highlighted. The table columns include Name, Instance ID, Instance state, Instance type, Status check, Alarm status, and Availability. The instance 'jenkins-server' has an ID of 'i-003e4f15b51851552' and is in a 'Running' state. The status check shows '3/3 checks passed'.

STEP 2: Install Required Software.

```
[root@ip-172-31-44-97 my-app]# docker --version
Docker version 25.0.14, build 0bab007
[root@ip-172-31-44-97 my-app]# jenkins --version
2.541.3
[root@ip-172-31-44-97 my-app]#
```

STEP 3: Create Dockerfile

The screenshot shows a terminal window with the title 'Dockerfile'. The content of the Dockerfile is as follows:

```
FROM eclipse-temurin:17
WORKDIR /app
COPY target/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

STEP 4: Create a Jenkinsfile to define pipeline stages.

Pipeline stages:

- Code Checkout
- Build Docker Image
- Push Image to Docker Registry
- Deploy Container

```
aws | Search | [Alt+S] | Ask Amazon Q | Asia Pacific (Mumbai) | Shreenivasa (0741-3904-4318) | Shreenivasa
```

```
GNU nano 8.3 Jenkinsfile
pipeline {
  agent any

  stages {
    stage('Checkout Code') {
      steps {
        git branch: 'main', url: 'https://github.com/srinivassindhanur7-dotcom/my-app.git'
      }
    }

    stage('Build Project') {
      steps {
        sh 'mvn clean package'
      }
    }

    stage('Build Docker Image') {
      steps {
        sh 'docker build -t my-app .'
      }
    }
  }
}
```

Help Write Out Where Is Cut Execute Location M-U Undo M-A Set Mark M-I To Bracket
Exit Read File Replace Paste Justify Go To Line M-E Redo M-G Copy M-E Where Was

STEP 4: Configure Jenkins.

← → ↻ ⚠ Not secure 13.201.71.172:8080/login?from=%2F



Sign in to Jenkins

Username

Password

☐ Keep me signed in

Sign in

Jenkins / docker-pipeline / Configure

Configure

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/srinivassindhanur7-dotcom/my-app.git

Credentials ?

- none -

+ Add

Advanced

Save Apply

STEP 5: Built the image successfully in Jenkins

Jenkins / docker-pipeline / #7

```
[Pipeline] sh
+ docker stop my-app
Error response from daemon: No such container: my-app
+ true
[Pipeline] sh
+ docker rm my-app
Error response from daemon: No such container: my-app
+ true
[Pipeline] sh
+ docker run -d -p 8081:8081 --name my-app my-app
9dd230884f08d31f53eb5b7a5908af44b4581e88f12bb787a5ded68f8d37de09
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

STEP 8: Successfully built the pipeline job.

Jenkins / docker-pipeline / Stages

Stages

15 April 2026

#12

09:56 - 29s

Start

Checkout SCM

Checkout...

Build Project

Build Docker..

Run Container

End

0.86s

0.67s

22s

1s

2s

#11

09:56 - 16s

Start

Checkout SCM

Checkout...

Build Project

Build Docker..

Run Container

End

1s

0.84s

7s

2s

2s

#10

09:55 - 14s

Start

Checkout SCM

Checkout...

Build Project

Build Docker..

Run Container

End

0.8s

0.59s

6s

2s

2s

Build

...

Jenkins / docker-pipeline

Status

docker-pipeline

Add description

Changes

Build Now

Configure

Delete Pipeline

Stages

Rename

Pipeline Syntax

Permalinks

Last build (#12), 7 min 22 sec ago

Last stable build (#12), 7 min 22 sec ago

Last successful build (#12), 7 min 22 sec ago

Last completed build (#12), 7 min 22 sec ago

Builds >

Filter

Today

#12

4:26 am

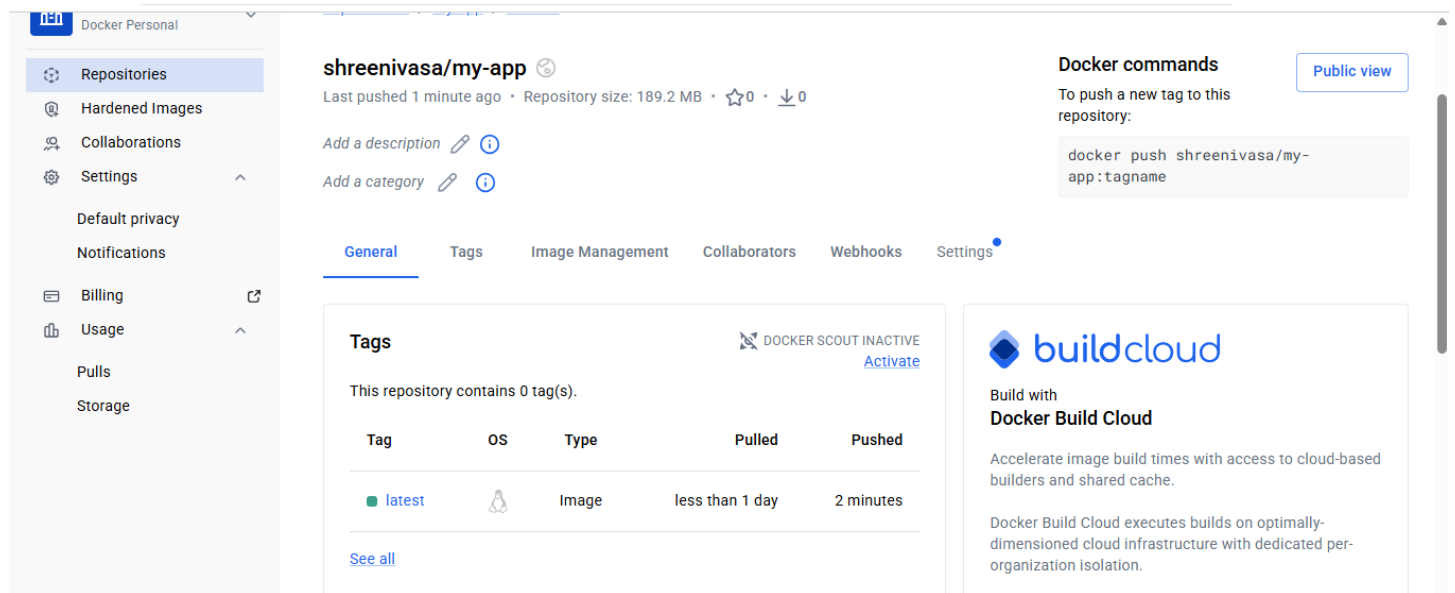
#11

4:26 am

#10

4:25 am

STEP 7: Image is pushed to the DockerHub.



The screenshot shows the Docker Hub interface for a repository named 'shreenivasa/my-app'. The left sidebar contains navigation links: Repositories, Hardened Images, Collaborations, Settings, Default privacy, Notifications, Billing, Usage, Pulls, and Storage. The main content area shows the repository details, including the name 'shreenivasa/my-app', last pushed time '1 minute ago', repository size '189.2 MB', and star/download counts. There are links to 'Add a description' and 'Add a category'. Below this is a tabbed interface with 'General' selected, showing a 'Tags' section with a table of repository tags. To the right of the tags is a 'DOCKER SCOUT INACTIVE' button with an 'Activate' link. Further right is a 'Docker commands' section with a 'Public view' button and a code block containing the command 'docker push shreenivasa/my-app:tagname'. At the bottom right is a 'buildcloud' advertisement for Docker Build Cloud.

shreenivasa/my-app
Last pushed 1 minute ago · Repository size: 189.2 MB · ☆ 0 · ↓ 0

[Add a description](#) [Add a category](#)

Tags DOCKER SCOUT INACTIVE [Activate](#)

This repository contains 0 tag(s).

Tag	OS	Type	Pulled	Pushed
latest		Image	less than 1 day	2 minutes

[See all](#)

Docker commands [Public view](#)

To push a new tag to this repository:

```
docker push shreenivasa/my-app:tagname
```

buildcloud
Build with **Docker Build Cloud**
Accelerate image build times with access to cloud-based builders and shared cache.
Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Conclusion: The AWS EC2 Auto Deployment using Docker project successfully demonstrates how cloud computing, containerization, and CI/CD automation can be integrated to streamline application deployment. By using Amazon EC2, Docker, and Jenkins, the project automates the process of building, storing, and deploying applications. The CI/CD pipeline ensures that any changes made in GitHub are automatically built into Docker images and deployed on the EC2 server. This approach reduces manual effort, minimizes deployment errors, and improves development efficiency. Overall, the project highlights the importance of DevOps practices in modern cloud-based application deployment.